CS402/MA492 Cryptography

Tobias Rossmann
School of Mathematics, Statistics and Applied Mathematics
NUI Galway

Lecture 14:

Cryptography in Python, part 4

Topics

- LFSRs as cryptosystems.
- Breaking a 1-dimensional affine cipher (see A1/Q3).
- Known-plaintext attacks on Vigenère ciphers (see A1/Q4).

Setup

In [1]:

```
from cs402 import *
```

In [2]:

```
%matplotlib inline
```

LFSRs as cryptosystems

We need to specify an LFSR. Keys are its initial states.

In [3]:

```
L = LFSR(4, [1,3,4])
S = StreamCipher(L)
```

In [4]:

```
plain = random_vector(24)
plain
```

Out[4]:

```
[1, 0, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0,
1, 1]
```

In [5]:

```
S.encrypt_bit_string([0,0,0,0], plain)
```

Out[5]:

```
[1, 0, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0,
1, 1]
```

In [6]:

```
S.encrypt_bit_string([1,0,0,0], plain)
```

Out[6]:

```
[0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 0, 0,
0, 0]
```

In [7]:

```
S.encrypt_bit_string([1,0,0,0], 24*[0])
```

Out[7]:

```
[1, 0, 0, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0,
1, 1]
```

Breaking an affine cipher

> An affine cryptosystem on single letter message units over the above 68-symbol alphabet was used to produce the ciphertext in the file a1q3-cipher7.txt (http://www.maths.nuigalway.ie/~rossmann/cs402/a1q3-cipher7.txt) Find the encryption key that was used to produce the ciphertext.

Read the ciphertext

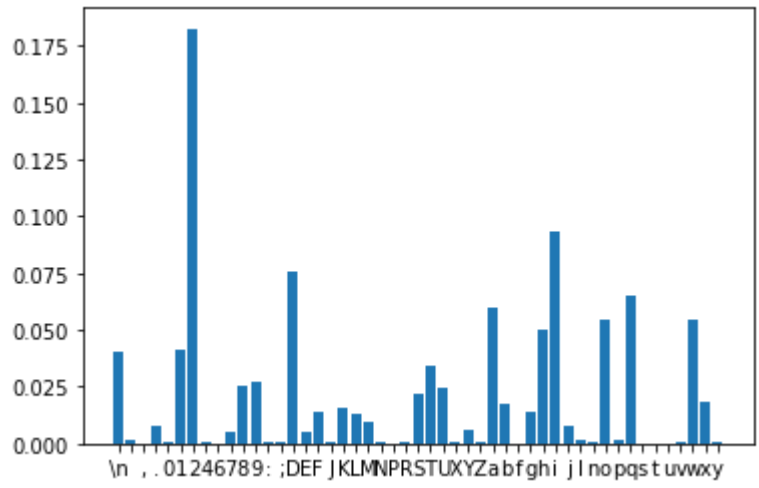
In [8]:

```
f = open('a1q3-cipher7.txt')
cipher = f.read()
f.close()
```

Frequency analysis

In [9]:

```
frequency_histogram(cipher)
```



We suspect:

Plaintext	Ciphertext
␣	2
e	i
t	D

Let's translate this into numbers:

In [10]:

```
string_to_int_list(ALPHABET68, ' et')
```

Out[10]:

```
[66, 4, 19]
```

In [11]:

```
string_to_int_list(ALPHABET68, '2iD')
```

Out[11]:

```
[54, 8, 29]
```

Hence, our suspicion is:

Plaintext	Ciphertext
66	54
4	8
19	29

We thus seek to find $a, b \in \mathbb{Z}_{68}$ with

$$\begin{aligned} 66a + b &\equiv 54 \pmod{68} \\ 4a + b &\equiv 8 \pmod{68} \\ 19a + b &\equiv 29 \pmod{68} \end{aligned}$$

and such that $\gcd(a, 68) = 1$(!).

The difference between the first two congruences yields

$$62a \equiv 46 \pmod{68}.$$

In [12]:

```
gcd(62,68)
```

Out[12]:

2

In general, such congruences (with gcd > 1) cannot be solved uniquely!

In [13]:

```
for a in range(68):
    if 62 * a % 68 == 46:
        print(a)
```

15
49

In [14]:

```
gcd(15,68)
```

Out[14]:

1

In [15]:

```
gcd(49,68)
```

Out[15]:

1

We could simply try out both candidates for a. Alternatively, let's use the third congruence.

If

$$\begin{aligned} 66a + b &\equiv 54 \pmod{68} \\ 19a + b &\equiv 29 \pmod{68} \end{aligned}$$

then

$$47a \equiv 25 \pmod{68}.$$

In [16]:

```
gcd(47,68)
```

Out[16]:

1

In [17]:

```
(modular_inverse(47,68) * 25) % 68
```

Out[17]:

15

In [18]:

```
gcd(15,68)
```

Out[18]:

1

Hence: a is (probably) 15.

Plug this into one of our congruences:

$$66a + b \equiv 66 \cdot 15 + b \equiv 54 \pmod{68}.$$

In [19]:

```
(54 - 66 * 15) % 68
```

Out[19]:

16

We thus suspect that $(a, b) = (15, 16)$. Let's to decrypt `cipher` :

In [ ]:

```
A68 = AffineCipher(ALPHABET68)
decrypted = A68.decrypt_string((15,16),cipher)
print(decrypted)
```

Digraph frequencies

Hint. For A2/Q2, use “random English texts” to find the most frequently used digraphs over `ALPHABET27` .

In [21]:

```
digraph_ranking(decrypted)
```

```
Most frequently used digraphs:
 1. "e "         100
 2. "th"          87
 3. " a"          80
 4. " t"          67
 5. "  "          66
 6. "s "          64
 7. "t "          62
 8. "at"          55
 9. "he"          51
10. " i"          51
```

Known-plaintext attacks on a Vigenère cipher

> A Vigenère cipher over the above 27-symbol alphabet was used to produce the ciphertext in the file a1q4-cipher4.txt (http://www.maths.nuigalway.ie/~rossmann/cs402/a1q4-cipher4.txt). The plaintext begins:
> I HAD BETTER SAY SOMETHING HERE ABOUT THIS
>
> Find the encryption key that was used to produce the ciphertext

In [22]:

```
f = open('a1q4-cipher4.txt')
cipher = f.read()
f.close()
```

In [23]:

```
plain_sample = 'I HAD BETTER SAY SOMETHING HERE ABOUT THIS'
len(plain_sample)
```

Out[23]:

```
42
```

In [24]:

```
int_plain_sample = string_to_int_list(ALPHABET27, plain_sample)
int_cipher_sample = string_to_int_list(ALPHABET27, cipher[:len(plain_sample)])
```

In [25]:

```
for i in range(42):
    print((int_cipher_sample[i] - int_plain_sample[i]) % 27, end=' ')
```

```
10 1 1 13 14 5 10 1 1 13 14 5 10 1 1 13 14 5 10 1 1 13 14 5 10 1 1 1
3 14 5 10 1 1 13 14 5 10 1 1 13 14 5
```

In [26]:

```
int_key = [10,1,1,13,14,5]
key = int_list_to_string(ALPHABET27, int_key)
key
```

Out[26]:

```
'KBBNOF'
```

In [ ]:

```
V = VigenereCipher(ALPHABET27)
V.decrypt_string(key, cipher)
```